Sesión 11

Modelos Complementarios:
Técnicas especializadas para problemas específicos

Carlos César Sánchez Coronel

MACHINE LEARNING

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Regresión polinómica

Idea

Añadir términos polinómicos de las variables originales como nuevas características para capturar relaciones no lineales.

Para una variable x, grado d:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d + \varepsilon$$

Sigue siendo un modelo **lineal en los parámetros** $\beta \rightarrow$ se estima por mínimos cuadrados.

Riesgo de sobreajuste

- A mayor grado d, mayor flexibilidad.
- Riesgo de **overfitting**, especialmente en los extremos.
- Se elige el grado mediante validación cruzada.

Ejemplo (Pregunta 2)

RMSE en validación: grado1=4.2, grado2=3.1, grado3=2.8, grado4=2.7, grado5=3.5.
Se elige grado 3 o 4 (trade-off sesgo-varianza).

- Regularización (Ridge, Lasso) permite grados altos sin sobreajuste.

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Regresión de Huber

Función de pérdida

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2 & \text{si } |r| \leq \delta \\ \delta(|r| - \frac{1}{2}\delta) & \text{en otro caso} \end{cases}$$

donde $r = y - \hat{y}$.

- Combina MSE (para errores pequeños) y MAE (para errores grandes).
- Menos sensible a outliers que MSE, más eficiente que MAE.
- Parámetro δ controla la transición (típico $\delta = 1{,}35$ da 95 % eficiencia si los errores son normales).

RANSAC (RANdom SAmple Consensus)

Algoritmo

1. Seleccionar aleatoriamente una muestra mínima para ajustar un modelo.
2. Calcular cuántos puntos están dentro de un umbral de distancia (inliers).
3. Repetir N iteraciones, guardar el modelo con más inliers.
4. Reajustar el modelo final con todos los inliers.

- Extremadamente robusto (tolera hasta ¿50 % de outliers).
- Parámetros: tamaño de muestra mínimo, umbral de distancia, número de iteraciones.

Comparación de pérdidas

Función	**Ecuación**	**Sensibilidad a outliers**
MSE	r^2	Alta
MAE	$\lvert r \rvert$	Moderada
Huber	$L_\delta(r)$	Baja (con δ adecuado)

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

SVR: formulación

Objetivo

Encontrar una función $f(x) = w^T x + b$ que tenga una desviación máxima ε respecto a los valores reales, y sea lo más plana posible.

Primal

$$\text{mín}\ \frac{1}{2}\|w\|^2 + C\sum(\xi_i + \xi_i^*)$$

Sujeto a

$$\begin{aligned} y_i - f(x_i) &\leq \varepsilon + \xi_i \\ f(x_i) - y_i &\leq \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* &\geq 0 \end{aligned}$$

Interpretación de parámetros

- ε: ancho de la banda de tolerancia (errores menores no penalizan).
- C: regularización; controla penalización por puntos fuera de la banda.
- Kernel: permite relaciones no lineales (RBF, polinomial, etc.).

SVR con kernel RBF

- γ (gamma) controla la influencia de cada punto.
- Valores grandes de γ pueden causar sobreajuste.
- Se ajustan con validación cruzada.

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

DBSCAN: elección de parámetros

- **minPts**: regla heurística $\geq$ dimensionalidad + 1 (típico 3-10).
- **eps (ε)**: se elige mediante el gráfico de distancia al k-ésimo vecino (k = minPts).
  1. Calcular distancia al k-ésimo vecino para cada punto.
  2. Ordenar y graficar.
  3. Elegir eps en el "codo" de la curva.

Variantes de DBSCAN

OPTICS No requiere un valor fijo de ε; produce un diagrama de alcanzabilidad que revela estructuras de densidad variable.

HDBSCAN Extensión jerárquica que encuentra clusters de diferentes densidades y es más robusta a los parámetros.

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Tipos de enlace

Single linkage

- Distancia mínima entre clusters.
- Tiende a formar cadenas (efecto "encadenamiento").

Complete linkage

- Distancia máxima.
- Clusters compactos y esféricos.

Average linkage

- Distancia promedio.
- Compromiso.

Ward linkage

- Minimiza incremento de varianza intra-cluster.
- Clusters de tamaño similar.

Corte del dendrograma

- Se corta a una altura donde las distancias de fusión son grandes.
- La elección puede basarse en:
  - Criterio visual (saltos grandes).
  - Silueta promedio para diferentes cortes.
- Complejidad computacional: $O(n^3)$ (ingenuo) o $O(n^2 \log n)$ con optimizaciones.

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Índice de Rand y ARI

- **Rand Index (RI)**: mide concordancia entre dos particiones.

$$RI = \frac{a+b}{\binom{n}{2}}$$

  donde $a =$ pares en mismo cluster en ambas, $b =$ pares en diferentes clusters en ambas.
- **Adjusted Rand Index (ARI)**: corrige por el azar, dando 0 para particiones aleatorias, 1 para coincidencia perfecta.

Información Mutua (MI) y NMI

- **MI**: mide la información compartida entre dos particiones.

$$MI(U, V) = \sum_{i,j} P(i,j) \log \frac{P(i,j)}{P(i)P(j)}$$

- **NMI (Normalized MI)**: escala a $[0, 1]$ dividiendo por la media de las entropías.

Comparación de métricas de clustering

Métrica	**Rango**	**Corrige azar**	**Requiere etiquetas**
Silueta	$[-1, 1]$	No	No
Davies-Bouldin	$[0, \infty)$ (menor mejor)	No	No
Rand Index	$[0, 1]$	No	Sí
ARI	$[-1, 1]$	Sí	Sí
NMI	$[0, 1]$	Aproximadamente	Sí

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Cuándo usar cada técnica

Técnica	**Cuándo usarla**
Regresión polinómica	Relación no lineal suave, pocas variables, interpretabilidad.
Regresión robusta (Huber)	Outliers moderados, se desea eficiencia.
RANSAC	Gran cantidad de outliers (hasta ¿50 %), modelo de inliers.
SVR	Relaciones no lineales, robustez a outliers, control sobre banda de error.
DBSCAN	Clusters de forma arbitraria, presencia de ruido, no se conoce k.
Clustering jerárquico	Se necesita jerarquía interpretable, pocos datos.

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Problema

- Planta industrial con sensores de temperatura, presión, vibración (10,000 lecturas, sin etiquetas).
- Objetivos:
  1. Identificar modos de operación normal (clusters).
  2. Detectar anomalías (posibles fallos incipientes).
  3. Modelar relaciones esperadas entre sensores.

Pipeline propuesto

1. **DBSCAN** para encontrar clusters de operación normal (puntos densos). El ruido son anomalías candidatas.
2. Para cada cluster, ajustar una **regresión robusta** (Huber o RANSAC) modelando, ej., temperatura vs presión.
3. En tiempo real:
   - Si un punto no pertenece a ningún cluster normal $\rightarrow$ alerta.
   - Si el residuo de la regresión supera un umbral $\rightarrow$ alerta.

Elección de parámetros y validación

- DBSCAN: elegir minPts (ej. 5) y eps con gráfico de distancia al 5-vecino.
- Para clusters normales, regresión robusta (Huber) o polinómica si la relación es no lineal.
- Si se tienen pocos eventos de fallo conocidos, usar **ARI/NMI** para validar que los clusters capturan estados pre-falla.

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Regresión polinómica y robusta

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression, HuberRegressor, RANSACRegressor
from sklearn.pipeline import Pipeline

# Polin mica
model = Pipeline([
    ('poly', PolynomialFeatures(degree=3)),
    ('linear', LinearRegression())
])
model.fit(X, y)

# Huber
huber = HuberRegressor(epsilon=1.35)
huber.fit(X, y)

# RANSAC
ransac = RANSACRegressor(min_samples=50, residual_threshold=5.0, max_trials=100)
ransac.fit(X, y)
inlier_mask = ransac.inlier_mask_
```

SVR y DBSCAN

```
from sklearn.svm import SVR
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors

# SVR
svr = SVR(kernel='rbf', C=1.0, epsilon=0.1)
svr.fit(X_train, y_train)

# Gr fico de distancias para DBSCAN
neigh = NearestNeighbors(n_neighbors=5)
neigh.fit(X)
distances, _ = neigh.kneighbors(X)
k_dist = np.sort(distances[:, -1])
plt.plot(k_dist); plt.show()

# DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X)
```

Clustering jerárquico y métricas externas

```
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

# Jer rquico
Z = linkage(X, method='ward')
plt.figure(figsize=(10,5))
dendrogram(Z)
plt.show()

clusters = fcluster(Z, t=3, criterion='maxclust')

# M tricas externas (con etiquetas verdaderas y_true)
ari = adjusted_rand_score(y_true, clusters)
nmi = normalized_mutual_info_score(y_true, clusters)
```

Contenido

1. Regresión Polinómica
2. Regresión Robusta
3. Support Vector Regression (SVR)
4. DBSCAN a fondo
5. Clustering Jerárquico a fondo
6. Métricas adicionales para clustering
7. Guía de selección de técnicas
8. Caso integrado: Sensores industriales
9. Implementación en Python
10. Conexión y Resumen

Conexión con el resto del curso

- Regresión robusta y SVR complementan los modelos de regresión.
- DBSCAN y clustering jerárquico amplían opciones de clustering (K-Means).
- Métricas externas (ARI, NMI) permiten evaluar clustering con ground truth.

Lo que hemos aprendido

Conceptos clave

- Regresión polinómica: no linealidades, sobreajuste, regularización.
- Regresión robusta: Huber, RANSAC, comparación de pérdidas.
- SVR: banda ε, parámetros C, γ, kernel.
- DBSCAN: elección de ε y minPts, variantes OPTICS/HDBSCAN.
- Clustering jerárquico: tipos de enlace, dendrograma, corte.
- Métricas externas: ARI, NMI.
- Caso práctico: detección de anomalías en sensores industriales.

¡Gracias!